DEVELOPER SPECIFICATION

cds-validate & cds-agents

Two Open-Source SAP CAP Libraries

Full Design, Coding & Deployment Plan

Table of Contents

1. Project Overview
2. Repository & Folder Structure
3. Package 1 — cds-validate
   3.1 What To Build
   3.2 File-by-File Coding Plan
   3.3 Testing Plan
4. Package 2 — cds-agents
   4.1 What To Build
   4.2 Phase-by-Phase Coding Plan
   4.3 Testing Plan
5. Week-by-Week Timeline
6. Deployment Plan
7. How to Publish to npm

1. Project Overview

You are building two independent npm packages for the SAP CAP (Cloud Application Programming Model) ecosystem. Both packages solve problems that every SAP developer faces but no one has solved properly yet.

Package	What It Does	Who Needs It	Build Time
cds-validate	Auto-validates SAP CAP request data using Zod schemas	Every CAP developer	2 weeks
cds-agents	Auto-generates LangGraph AI tools from CDS service definitions	Any CAP dev adding AI	4–5 weeks

Build cds-validate FIRST. It is smaller, easier, and gets you community credibility before tackling cds-agents.

2. Repository & Folder Structure

2.1 Top-Level Monorepo Layout

Create a monorepo with two packages. Use a tool called pnpm workspaces to manage them.

Root folder name: cds-oss-libs

```
cds-oss-libs/
  packages/
    cds-validate/        ← Package 1
    cds-agents/          ← Package 2
  demo-app/              ← Real CAP app used for testing
  package.json           ← Monorepo root
  pnpm-workspace.yaml
  .github/workflows/     ← CI/CD pipelines
```

2.2 cds-validate Folder Structure

```
cds-validate/
  src/
    index.ts             ← Main entry — exports validate()
    validator.ts         ← Core middleware logic
    error-mapper.ts      ← Zod errors → OData error format
    schema-builder.ts    ← Auto-builds Zod schema from CDS model
    types.ts             ← TypeScript type definitions
  tests/
    validator.test.ts
    error-mapper.test.ts
    schema-builder.test.ts
  package.json
  tsconfig.json
  README.md
```

2.3 cds-agents Folder Structure

```
cds-agents/
  src/
    index.ts             ← Main entry — exports CAPAgent & CAPToolkit
    agent.ts             ← CAPAgent class
```

```
  tool-generator.ts    ← Reads CDS → creates LangGraph tools
  type-mapper.ts       ← CDS types → Zod types
  http-client.ts       ← Makes OData HTTP calls
  error-handler.ts     ← Parses OData errors
  odata-builder.ts     ← Builds OData query strings
  toolkit.ts           ← CAPToolkit class (raw tools without agent)
  providers/
    anthropic.ts       ← Claude model setup
    openai.ts          ← GPT-4o model setup
  types.ts             ← TypeScript interfaces
tests/
package.json
tsconfig.json
README.md
```

3. Package 1 — cds-validate

3.1 What To Build

A middleware function that a CAP developer wraps around their request handler. Instead of writing 50 lines of if-checks, they pass a Zod schema and this library does all the validation automatically.

The exported API surface (what the developer uses):

- validate(schema) — main middleware, validates req.data
- validateQuery(schema) — validates query parameters
- validateParams(schema) — validates action parameters
- schemaFromEntity(entityName) — auto-builds a Zod schema from a CDS entity

3.2 File-by-File Coding Plan

File: src/types.ts — Write This First

Define all TypeScript interfaces here before writing any logic. This file has no dependencies so it is the safest starting point.

- CAPRequest interface — shape of the SAP req object (data, query, params, error function)
- ValidationOptions interface — optional config like { strict: boolean, normalize: boolean }
- CAPError interface — shape of the OData error object

File: src/error-mapper.ts — Write Second

Takes a Zod validation result and converts it into a CAP-compatible error object.

- Input: Zod SafeParseError object
- Output: CAP OData error with code, message, target, and details array
- Target field: join the Zod error path array with forward slashes — this is already valid OData format
- Details array: map every Zod error into { message, target } pairs
- Use error code 422 (Unprocessable Entity) for validation failures

File: src/validator.ts — The Core

The main logic. This function returns another function (it is a factory/higher-order function).

- Call schema.safeParse(req.data)

- If it fails: call error-mapper, then call req.error() with the result — stop processing
- If it passes: optionally overwrite req.data with the parsed result (this gives you type coercion for free)
- For UPDATE events: automatically call schema.partial() so missing fields are not flagged as errors
- Detect the event type from req.event and apply partial() automatically

File: src/schema-builder.ts — Bonus Feature

Reads the CDS compiled model and returns a Zod schema for a named entity.

- Load the CDS model using cds.load() from @sap/cds
- Find the entity by name in the loaded model
- Loop through its elements (fields) and convert each type to Zod using the type map
- Mark required vs optional based on CDS @mandatory annotation
- Return the assembled z.object() schema

File: src/index.ts — Last

Just re-exports everything from the other files. Keep this file to 5–10 lines total.

3.3 Testing Plan for cds-validate

Test Case	What To Test	Expected Result
Valid data	Pass correct student object through validate()	req.data is set, no error called
Missing required field	Pass object without email	req.error called with target: 'email'
Wrong type	Pass string for GPA field	req.error called with type error message
UPDATE partial	Pass only one field on UPDATE event	No error — other fields ignored
Multiple errors	Pass object with 3 bad fields	details array has 3 entries
Data normalization	Pass email with spaces	req.data.email is trimmed
schemaFromEntity	Call with 'Students' entity name	Returns valid Zod schema object

4. Package 2 — cds-agents

4.1 What To Build

A library that reads your SAP CAP service definition, generates LangGraph tools for every entity and action automatically, and exposes them through a simple CAPAgent class that accepts plain English instructions.

The exported API surface:

- CAPAgent class — the main entry point, invoke() and stream() methods
- CAPToolkit class — just the raw tools, no agent wrapper, for advanced users

4.2 Phase-by-Phase Coding Plan

Phase 1 — src/type-mapper.ts

This is a pure lookup table. No HTTP calls, no CDS loading. Just a function that takes a CDS type string and returns the correct Zod type.

CDS Type	Zod Type	Notes
cds.String	z.string()	
cds.UUID	z.string().uuid()	
cds.Integer	z.number().int()	
cds.Decimal	z.number()	Use z.coerce.number() for safety
cds.Boolean	z.boolean()	
cds.Date	z.string()	Format: YYYY-MM-DD
cds.DateTime	z.string().datetime()	
cds.LargeString	z.string()	No max length
Unknown type	z.any()	Fallback — never crash

Also handle associations (linked entities) — map them to z.string().uuid().optional() since they arrive as foreign keys.

Phase 2 — src/odata-builder.ts

Builds OData v4 URL strings from plain inputs. This is a pure utility — no HTTP calls.

- buildReadUrl(baseUrl, service, entity, options) — builds GET URL with $filter, $select, $top, $skip, $orderby, $expand
- buildCreateUrl(baseUrl, service, entity) — builds POST URL
- buildUpdateUrl(baseUrl, service, entity, id) — builds PATCH URL with key predicate
- buildDeleteUrl(baseUrl, service, entity, id) — builds DELETE URL
- buildActionUrl(baseUrl, service, actionName) — builds POST URL for actions
- buildFilterString(conditions) — converts JS object to OData $filter syntax

The hardest part is buildFilterString. Support at minimum: eq, lt, gt, le, ge, ne operators. Accept an object like { gpa: { lt: 2.0 }, status: 'active' } and output gpa lt 2.0 and status eq 'active'.

Phase 3 — src/http-client.ts

Makes the actual HTTP calls to the CAP service. Wraps Node.js fetch with CAP-specific handling.

- Constructor accepts: baseUrl, auth config (basic or bearer token)
- get(url) — GET request, returns parsed JSON or throws
- post(url, body) — POST request
- patch(url, body) — PATCH request
- delete(url) — DELETE request
- All methods add Authorization header automatically based on auth config
- All methods check response.ok — if false, read the error body and throw a structured error
- Parse OData error format: error.message and error.details from the response body

Phase 4 — src/tool-generator.ts — The Most Important File

This file reads the CDS model and creates all the LangGraph tool definitions. This is the heart of the entire library.

Step A — Load the CDS model:

- Call cds.load() with the path to the srv/ folder
- Find the target service by name in csn.services
- If service not found, throw a clear error: 'Service StudentService not found in CDS model'

Step B — Generate CRUD tools for each entity:

- Loop over Object.entries(service.entities)
- For each entity, generate 4 tools using the LangGraph tool() function
- READ tool: input schema has optional filter, select, top (number), skip (number), orderBy fields

- CREATE tool: input schema mirrors the entity fields from type-mapper
- UPDATE tool: input schema has required id field + all entity fields as optional (partial update)
- DELETE tool: input schema has required id field only
- Each tool description must be specific, e.g.: 'Retrieves a list of Students. Use filter to narrow results by field values.'

Step C — Generate tools for custom actions:

- Loop over Object.entries(service.actions || {})
- Read the action's params from the CDS definition
- Build input schema from params using type-mapper
- Generate a single tool that POSTs to the action URL

Step D — Apply entity filtering:

- Accept an include array and exclude array in the config
- If include is set, only generate tools for those entities
- If exclude is set, skip those entities
- Always skip entities whose name starts with underscore (internal CAP entities)

Phase 5 — src/providers/anthropic.ts and openai.ts

Thin wrappers that create the LangGraph-compatible LLM object from config.

- anthropic.ts: takes model name and API key → returns ChatAnthropic instance from @langchain/anthropic
- openai.ts: takes model name and API key → returns ChatOpenAI instance from @langchain/openai
- Both export a createModel(config) function with identical signatures
- Default model names: claude-sonnet-4-5 for Anthropic, gpt-4o for OpenAI

Phase 6 — src/toolkit.ts

The CAPToolkit class — for advanced users who want the raw tools but will build their own LangGraph graph.

- Constructor: same config as CAPAgent
- async init() — loads CDS model and generates tools, must be called before getTools()
- getTools() — returns the raw array of LangGraph tool objects
- This is also what CAPAgent uses internally

Phase 7 — src/agent.ts — The Final Wrapper

The CAPAgent class that developers actually use. This should be the simplest file to write because all the hard work is done in the previous phases.

- Constructor accepts: service, baseUrl, model, provider, tools, exclude, auth, maxSteps
- async invoke(message) — calls the LangGraph ReAct agent with the message, returns final string response
- async stream(message) — same but returns an async iterator for streaming tokens
- Private method: createAgent() — loads toolkit, picks provider, creates ReAct agent from createReactAgent() in LangGraph
- Lazy initialization — do not load CDS or connect until first invoke() call
- maxSteps default: 10 — prevents infinite loops

4.3 Testing Plan for cds-agents

Test Case	What To Test	How To Test
Type mapper	All 9 CDS types convert correctly	Unit test — no CAP needed
OData builder	All 5 URL types build correctly	Unit test with string assertions
Filter builder	Complex filter object → correct string	Unit test with 6 combinations
Tool generation	5 entities → 20 CRUD tools created	Mock CDS model, check tool count
Action tools	Custom action appears as tool	Mock CDS model with 1 action
Entity filtering	exclude: ['Grades'] removes those tools	Unit test
HTTP GET	Read tool calls correct OData URL	Mock fetch, verify URL called
HTTP POST	Create tool sends correct body	Mock fetch, verify body
Error handling	404 from CAP → readable error to AI	Mock fetch returning 404
Full integration	agent.invoke() on real CAP app	Integration test with demo app

5. Week-by-Week Timeline

Week	Focus	Deliverable
Week 1	cds-validate: types, error-mapper, core validator	validate() works end-to-end
Week 2	cds-validate: schema-builder, tests, README, publish	v1.0.0 live on npm
Week 3	cds-agents: type-mapper, odata-builder, http-client	URL building + HTTP layer done
Week 4	cds-agents: tool-generator, toolkit class	Tools auto-generate from CDS
Week 5	cds-agents: agent class, providers, streaming	CAPAgent.invoke() works
Week 6	cds-agents: edge cases, demo app, README, publish	v1.0.0 live on npm

6. Deployment Plan

6.1 GitHub Repository Setup

1. Create two public GitHub repositories:
   - github.com/YOUR_USERNAME/cds-validate
   - github.com/YOUR_USERNAME/cds-agents
2. Add these files to each repo root:
   - README.md — usage docs with copy-paste examples
   - LICENSE — use MIT license
   - .gitignore — ignore node_modules, dist, .env
   - CHANGELOG.md — track versions
3. Set up branch protection: require PR review before merging to main

6.2 GitHub Actions CI/CD Pipeline

Create .github/workflows/ci.yml in each package. This runs automatically on every push.

The pipeline does 4 things in order:

4. Install dependencies using pnpm
5. Run TypeScript compiler to catch type errors
6. Run all tests using Jest
7. If on main branch AND a version tag was pushed: publish to npm automatically

To trigger a release, you just run: git tag v1.0.0 && git push --tags

6.3 Versioning Strategy

Follow Semantic Versioning (semver):

Version Change	When To Use	Example
Patch (1.0.X)	Bug fixes, no new features	1.0.0 → 1.0.1
Minor (1.X.0)	New features, backwards compatible	1.0.0 → 1.1.0

Version Change	When To Use	Example
Major (X.0.0)	Breaking changes to the API	1.0.0 → 2.0.0

6.4 Demo App Deployment

Build a simple web UI that demonstrates cds-agents. Deploy it free using:

- CAP backend: SAP BTP Free Tier or Railway.app
- Chat frontend: Vercel (free tier, deploys from GitHub automatically)
- The demo should show: type a question → see the AI call OData tools → get an answer

7. How to Publish to npm

Step 1 — Create an npm Account

- Go to npmjs.com and create a free account
- Verify your email address
- Enable 2-factor authentication (required for publishing)

Step 2 — Configure package.json

These fields are mandatory before publishing:

Field	What To Set	Example Value
name	Package name — must be globally unique on npm	cds-validate
version	Start at 1.0.0	1.0.0
description	One sentence description	Zod validation middleware for SAP CAP
main	Entry point for CommonJS	./dist/index.js
module	Entry point for ESM	./dist/index.mjs
types	TypeScript definitions	./dist/index.d.ts
files	What files to include in the package	["dist", "README.md"]
license	Must match LICENSE file	MIT
keywords	Helps people find your package	["sap", "cap", "cds", "validation"]
repository	Link to your GitHub repo	{ url: github.com/you/cds-validate }
peerDependencies	What the user must already have installed	@sap/cds, zod

Step 3 — Build the Package

Before publishing, compile TypeScript to JavaScript:

- Run tsc to compile TypeScript → JavaScript into the dist/ folder
- Run tsc-alias to fix path aliases if you used them
- Check that dist/ contains .js, .mjs, and .d.ts files

- Never publish TypeScript source — only compiled JavaScript + type definitions

Step 4 — Test the Package Locally

Before publishing to npm, test it locally so you don't publish a broken package:

- Run npm pack — this creates a .tgz file exactly like what npm will publish
- In your demo CAP project, run npm install ../path/to/cds-validate-1.0.0.tgz
- Test that import and usage works correctly
- Fix any issues before proceeding

Step 5 — Login and Publish

Run these commands in your terminal:

```
npm login
```

This opens a browser window — log in with your npm credentials.

```
npm publish --access public
```

The --access public flag is required for scoped packages (packages named like @yourname/cds-validate). For unscoped packages it is optional.

After publishing, verify it worked:

```
npm info cds-validate
```

You should see your package details printed in the terminal.

Step 6 — Update and Re-publish

Every time you want to release a new version:

8. Update CHANGELOG.md with what changed
9. Run npm version patch (or minor or major depending on the change)
10. This automatically bumps the version in package.json and creates a git tag
11. Run npm publish
12. Run git push && git push --tags to sync with GitHub

Step 7 — Post-Publish Checklist

- Add a badge to your README: ![npm](https://img.shields.io/npm/v/cds-validate)

- Post in SAP Community at community.sap.com
- Post on DEV.to with a tutorial article
- Submit a PR to the awesome-cap GitHub list
- Share on LinkedIn with a demo GIF

End of Specification Document